

Exponer Relay por MCP

Cómo se construyó el endpoint `/api/mcp`, qué expone y qué deja fuera a propósito.

Un recorrido para quien tiene que explicarlo en una llamada, no solo publicarlo.

1. Qué es MCP, en un párrafo

Model Context Protocol es una forma estándar de que una aplicación de IA descubra y llame herramientas que viven en otro lado. Antes, cada integración era a la medida: si querías que Claude Desktop leyera tus pedidos, alguien escribía un plugin para Claude, luego otro para Cursor, luego otro más. MCP lo reemplaza con un solo contrato — un servidor publica sus tools y cualquier cliente compatible las puede llamar.

💡 "MCP is a standard plug. We expose our tools once, and any AI tool that speaks it can use them — Claude Desktop, Cursor, or an agent someone builds next year."

2. Por qué expusimos Relay así

Relay ya tenía una capa de tools funcionando: nueve funciones con esquema, validación y auditoría, conectadas a una base de datos viva. Esas tools solo se alcanzaban desde el chat de Relay.

Exponerlas por MCP cuesta unas cuarenta líneas y cambia la oferta:

- El cliente sigue trabajando en la herramienta que ya usa. Sin pestaña nueva, sin login nuevo.
- Las tools dejan de ser una función de nuestro producto y se vuelven **su** capacidad.
- Cualquier cliente MCP futuro funciona sin que nosotros publiquemos nada.

La versión comercial de esa frase:

💡 "You don't have to move into our interface. We can put these tools inside the one your team already has open."

3. La decisión de diseño que importa

La capa de tools tiene ocho de lectura y una de escritura. **Solo se exponen las de lectura.**

`issue_refund` está ausente, y no porque fuera difícil. La compuerta de aprobación humana vive dentro de la interfaz de Relay — la tarjeta que muestra el monto y espera a que alguien apruebe. Si la tool de reembolso fuera alcanzable por MCP, un cliente externo podría llamarla directo y la tarjeta nunca se renderizaría. La propiedad de seguridad desaparecería, y nada en el protocolo lo habría impedido.

Así que la frontera se dibuja por lo que existe en cada borde, no por configuración:

Dónde	Lecturas	Escritura
Chat propio de Relay	sí	sí, detrás de la compuerta
Especialistas (subagentes)	sí	no — nunca se delega
Endpoint MCP	sí	no — nunca se expone

Tres lugares distintos, una sola regla: **la escritura solo existe donde está la compuerta.**

💡 "Reads are safe to federate. The write isn't, so it stays where the human approval lives. That's not a setting — the tool simply isn't there."

4. Cómo se construyó

Un solo archivo de ruta. Las tools ya existían, así que el trabajo fue adaptarlas al protocolo.

4.1 Reutilizar las tools existentes

Las definiciones ya son `{ description, inputSchema, execute }`. El servidor MCP registra cada una:

```
// src/app/api/mcp/route.ts
import { createMcpHandler } from "mcp-handler";
import { createBusinessTools, createKnowledgeTools } from "@lib/tools";

// Solo tools de lectura. La ruta de escritura está ausente a propósito.
const EXPOSED = { ...createBusinessTools(SESSION), ...createKnowledgeTools(SESSION) };
```

Fíjate en lo que **no** se importa: `createWriteTools`. Esa única omisión es la frontera de seguridad. Es una línea, es greppable, y un revisor la verifica en cinco segundos — que es justo la propiedad que quieres en un control de seguridad.

4.2 Registrar cada tool

```
const handler = createMcpHandler((server) => {
  for (const [name, definition] of Object.entries(EXPOSED)) {
    server.registerTool(
      name,
      { description: tool.description, inputSchema: tool.inputSchema },
      async (input) => {
        const result = await tool.execute(input, {});
        return {
          content: [
            { type: "text", text: result?.summary ?? "No result." },
            { type: "text", text: JSON.stringify(result, null, 2) },
          ],
        };
      },
    );
  }
});
```

Dos bloques de contenido a propósito: primero el resumen de una línea, para que un cliente que solo renderiza texto igual muestre algo legible, y luego el payload estructurado completo para el que sí lo puede usar.

4.3 Instrucciones para el modelo que se conecta

Un servidor MCP puede enviar instrucciones que el cliente le pasa a su modelo. Las de Relay llevan la misma regla de fundamentación que tienen los agentes internos:

"Every fact you report must come from a tool result — do not infer or invent order numbers, amounts or policy rules. Write actions are not available here by design."

Sin esto, un modelo conectado desde fuera no tiene nada de la disciplina de prompt de Relay y va a rellenar huecos con gusto.

4.4 Exportar como métodos HTTP

```
export { handler as GET, handler as POST, handler as DELETE };
```

MCP sobre HTTP en streaming usa los tres: POST para llamadas, GET para el stream de eventos, DELETE para cerrar sesión.

5. Conectar un cliente

```
// claude_desktop_config.json
{
  "mcpServers": {
    "relay": { "url": "http://localhost:3000/api/mcp" }
  }
}
```

Reinicias el cliente y las tools aparecen.

6. Verificarlo sin cliente

Dos llamadas de curl prueban que el endpoint funciona. Útil en una llamada cuando no quieres reiniciarle la app a nadie.

Listar las tools:

```
curl -s -X POST http://localhost:3000/api/mcp \
-H 'content-type: application/json' \
-H 'accept: application/json, text/event-stream' \
-d '{"jsonrpc":"2.0","id":1,"method":"tools/list"}'
```

Regresa las ocho de lectura:

```
find_delayed_orders · get_customer · get_operations_summary · get_order  
search_knowledge · search_orders · search_tickets · track_shipment
```

Llamar una:

```
curl -s -X POST http://localhost:3000/api/mcp \  
-H 'content-type: application/json' \  
-H 'accept: application/json, text/event-stream' \  
-d '{"jsonrpc":"2.0","id":2,"method":"tools/call",  
  "params":{"name":"get_order","arguments":{"orderNumber":"HP-1042"}}}'
```

Regresa:

```
"HP-1042 – fulfilled, $537.00, Danielle Okafor"
```

La verificación que importa — confirmar que la tool de escritura no se alcanza:

```
curl -s -X POST http://localhost:3000/api/mcp \  
-H 'content-type: application/json' \  
-H 'accept: application/json, text/event-stream' \  
-d '{"jsonrpc":"2.0","id":1,"method":"tools/list"}' | grep -c issue_refund  
# → 0
```

Corre esa frente a un cliente preocupado por seguridad. Una ausencia demostrada vale más que un párrafo asegurándola.

7. Lo que no construimos, y por qué

Nada de generar MCP desde OpenAPI. Convertir una especificación arbitraria en un servidor MCP generado es otro producto, con una superficie mucho mayor: intermediación de auth, custodia de credenciales, versionado, publicación. Relay expone sus propias tools, que ya valida y audita.

Sin autenticación todavía. El endpoint está abierto, que es correcto para una demo en localhost e incorrecto para producción. `mcp-handler` trae `withMcpAuth` para verificar tokens bearer; un despliegue real envuelve el handler ahí y acota los tokens por inquilino. Dilo derecho si un cliente pregunta — asegurar que una demo está endurecida para producción es la forma más rápida de perder la sala.

Ninguna tool de escritura, nunca, sin compuerta. Si un cliente pide escritura por MCP, la respuesta no es "se la exponemos" — es que el paso de aprobación tiene que moverse con ella. En algún lado un humano sigue apretando el botón.

8. Dónde ver el código

Archivo	Qué tiene
src/app/api/mcp/route.ts	El servidor MCP completo, unas cuarenta líneas
src/lib/tools.ts	Las definiciones de tools, compartidas con los agentes
src/lib/audit.ts	El registro — las llamadas MCP se auditan como cualquier otra

Las llamadas que llegan por MCP caen en la misma bitácora que las del chat, bajo el id de sesión `mcp`. Abre la pestaña Auditoría después de que un cliente conectado use una tool y ahí está.